Build a Digital Twin for a Potted Plant

What you'll build. A ~70-line Python script, `twin.py`, that simulates a potted plant losing moisture over 24 hours and a digital twin that reads its moisture sensor, predicts when the soil will dry out, and waters the plant automatically – no human in the loop. Running it prints a line per hour and, partway through, one marked `WATERED`:

```
hour 11  sensor= 38.2  predicted_next= 35.0
hour 12  sensor= 36.3  predicted_next= 34.4  WATERED
hour 13  sensor= 59.3  predicted_next= 82.3
```

What you'll learn. How to structure a digital twin as two independent pieces – a physical-side simulation and a twin that only ever sees sensor readings – and why closing the loop (the twin issuing a command back to the physical side) is what separates a digital twin from a dashboard.

What you need. Python 3.10 or later, and a terminal. No packages beyond the standard library.

Time. About 20 minutes.

Step 1: Create the project folder

```
mkdir plant-twin
cd plant-twin
```

Step 2: Simulate the pot

You don't have a real plant wired up, so start by writing a stand-in for one. Create `twin.py` with this content:

```python
import random

random.seed(7)


class Pot:
    """Stands in for the real pot's soil -- in a hardware deployment this
    state lives in wet dirt, not a Python object."""

    def __init__(self, moisture=60.0):
        self.moisture = moisture

    def tick(self):
        self.moisture -= random.uniform(1.5, 2.5)
        self.moisture = max(0.0, self.moisture)

    def water(self):
        self.moisture = min(100.0, self.moisture + 25.0)
```

```
    def read_sensor(self):
        return round(self.moisture + random.uniform(-1.0, 1.0), 1)
```

`tick()` is one hour passing: the soil loses a random amount of moisture, the way real soil does under evaporation. `read_sensor()` is what a real capacitive soil-moisture probe gives you – the true value, plus a little sensor noise. `random.seed(7)` pins that randomness so your output matches this tutorial's exactly.

Check that it works before moving on:

```
python3 -c "
from twin import Pot
pot = Pot()
for _ in range(3):
    pot.tick()
    print(pot.read_sensor())
"
```

You should see:

```
57.5
55.2
53.7
```

Three ticks, three shrinking readings. That's the physical side done – notice it knows nothing about digital twins. It's just a pot.

Step 3: Write the digital twin

The twin is a second, separate class. Critically, it never touches `Pot` directly – it only ever sees numbers arriving from `read_sensor()`, the same way a real twin only sees whatever its sensor's network connection delivers. Append this to `twin.py`:

```
class PlantTwin:
    """The digital twin. It never touches Pot directly -- only sensor
    readings flow in, and water commands flow back out."""

    DRY_THRESHOLD = 35.0

    def __init__(self):
        self.history = []

    def ingest(self, reading):
        self.history.append(reading)
        self.history = self.history[-3:]

    def predict_next(self):
        if len(self.history) < 2:
            return self.history[-1]
        trend = self.history[-1] - self.history[-2]
```

```
        return self.history[-1] + trend

    def decide(self):
        return self.predict_next() < self.DRY_THRESHOLD
```

`ingest()` is the twin's half of the data connection: each new reading gets appended, and only the last three are kept, since only the recent trend matters for what comes next. `predict_next()` is the twin's model – deliberately the simplest one that works: extend the line between the last two readings one more hour forward. `decide()` is the twin's algorithm: if that predicted value would be too dry, it's time to water. This is exactly the three-part shape every digital twin shares – data in, a model to interpret it, an algorithm to act on it – just small enough here to read in one sitting.

Step 4: Close the loop

A twin that only predicts is a dashboard. What makes it a *twin* is that its decision changes the physical side. Append the loop that wires `Pot` and `PlantTwin` together:

```
def run(hours):
    pot = Pot()
    twin = PlantTwin()

    for hour in range(1, hours + 1):
        pot.tick()
        reading = pot.read_sensor()
        twin.ingest(reading)

        watered = False
        if twin.decide():
            pot.water()
            watered = True

        status = "WATERED" if watered else ""
        print(f"hour {hour:2d}  sensor={reading:5.1f}  "
              f"predicted_next={twin.predict_next():5.1f}  {status}")


if __name__ == "__main__":
    run(24)
```

Each hour, the pot's real moisture drops, the sensor reports a noisy version of it, the twin folds that reading into its prediction, and if the twin's prediction crosses the dry threshold, it calls `pot.water()` – reaching back across the loop to change the physical side, before a human ever looks at the readings.

Step 5: Run it

```
python3 twin.py
```

You should see 24 lines of output. Line 12 is marked `WATERED`, and the next line's `sensor` value jumps back up above 59, because the twin's command already changed the pot before that reading was taken:

```
hour 11  sensor= 38.2  predicted_next= 35.0
hour 12  sensor= 36.3  predicted_next= 34.4  WATERED
hour 13  sensor= 59.3  predicted_next= 82.3
```

If your output matches, the loop is genuinely closed: the twin's decision at hour 12 is the reason hour 13's real-world reading looks completely different.

Step 6: Change the twin's behavior without touching the pot

Open `twin.py` and change:

```
DRY_THRESHOLD = 35.0
```

to:

```
DRY_THRESHOLD = 45.0
```

Save the file and run `python3 twin.py` again. Watering now happens at hour 8 instead of hour 12, and a second time at hour 21:

```
hour  8  sensor= 45.2  predicted_next= 42.3  WATERED
...
hour 21  sensor= 44.5  predicted_next= 42.0  WATERED
```

Notice what you didn't have to change: `Pot` is untouched. This is the payoff of keeping the twin as a separate piece that only talks to the physical side through readings and commands – you can retune, replace, or completely rewrite the twin's model without ever touching the thing it's twinning.

What you built

You now have a working digital twin: a physical-side simulation that knows nothing about twins, a twin that knows nothing about the physical side's internals, and a closed loop connecting them where the twin's own prediction changes what the physical side does next. Swap `Pot` for code that reads a real capacitive soil sensor over I2C and calls a relay to run a water pump, and the `PlantTwin` class doesn't need to change at all – that's the whole reason to build the two halves this way.

Where to go next

`PlantTwin`'s "extend the last two points" model is deliberately the crudest thing that could work. A real deployment would replace `predict_next` with a Kalman filter, which blends the model's prediction against each new noisy reading instead of trusting the latest one alone [1], and would read real sensor data over MQTT rather than calling a Python method directly. A how-to guide would be the right place to cover wiring up an actual moisture probe and pump, since that involves choices – which board, which protocol – that a first tutorial deliberately avoids.

Further reading

[1] H. Feng, C. Gomes, and P. G. Larsen, *Model-Based Monitoring and State Estimation for Digital Twins: The Kalman Filter*, arXiv, 2023. `feng_model-based_2023`

[1] H. Feng, C. Gomes, and P. G. Larsen, "Model-Based Monitoring and State Estimation for Digital Twins: The Kalman Filter." arXiv, Apr. 2023. doi: 10.48550/arXiv.2305.00252.